

Table Relationships, Joins, and pandas Loading

A one-block lesson on moving relational data into analysis-ready DataFrames

Appendix E.2

Use SQL joins to combine related tables, then load the useful result directly into pandas.



Mental model: SQL shapes the dataset; pandas analyzes and communicates it.

Block Plan

30 minutes instruction + 30 minutes guided practice

Related tables

Keys

INNER JOIN

LEFT JOIN

pandas load

Exercises

Instruction

- Why multiple tables exist
- Primary keys and foreign keys
- INNER JOIN and LEFT JOIN
- Loading joined results into pandas

Guided practice

- Inspect both tables
- Write joins with aliases
- Compare join types
- Load with `read_sql_query()`

Applied output

- Notebook report section
- Joined DataFrame
- Missing source review
- Pandas summary + chart

Big idea: ask SQLite for the useful joined dataset, then continue the analysis in pandas.

Why Use More Than One Table?

Separate repeated source details from repeated activity rows

record_id	source_name	department	owner
EQ-1001	maintenance	operations	ops@...
EQ-1002	logistics	supply	supply@...
EQ-1003	maintenance	operations	ops@...



Repeated values create update risk

Relational design

source_systems
one row per source

equipment_activity
one row per event

Store each kind of fact in the table where it belongs.

The Related Tables

source_id connects activity rows to source metadata

source_id	source_name	department	activity_id	record_id	source_id	status	record_count
1	maintenance	operations	1	EQ-1001	1	ready	12000
2	logistics	supply	2	EQ-1002	2	warning	8500
3	finance	finance	3	EQ-1003	1	ready	6400

The activity table stores source_id. The source table stores the human-readable source details.

Primary Keys and Foreign Keys

One key identifies; another key refers

Primary key

source_systems.source_id

Uniquely identifies one row in its own table.



Foreign key

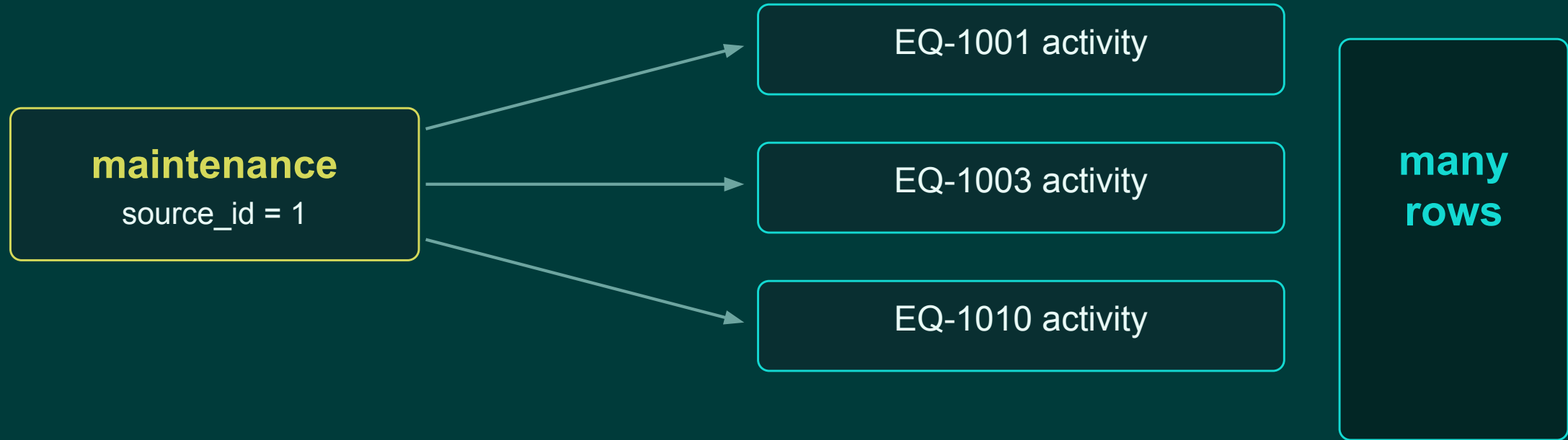
equipment_activity.source_id

Stores the identity of a related row in another table.

Join condition: activity.source_id = sources.source_id

One-to-Many Relationship

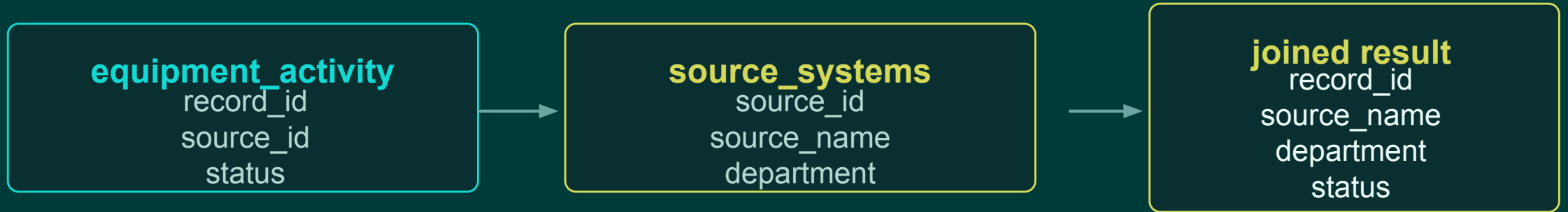
One source system can have many activity rows



The source details are stored once, then referenced by many activity records.

The Purpose of a Join

Combine related rows for one query result



```
ON activity.source_id = sources.source_id
```

A join does not permanently merge tables. It creates a useful result for this query.

First INNER JOIN

Return rows that match in both tables

```
SELECT
    act.record_id,
    src.source_name,
    act.status,
    act.record_count
FROM equipment_activity AS act
INNER JOIN source_systems AS src
    ON act.source_id = src.source_id;
```

Mental model

- Start with activity rows
- Find a matching source row
- Combine selected columns
- Return matched rows only

INNER JOIN is appropriate when only fully matched data should appear in the result.

Table Aliases and Qualified Columns

Make joined queries readable and unambiguous

```
SELECT
    act.record_id,
    src.source_name,
    act.status,
    act.record_count
FROM equipment_activity AS act
INNER JOIN source_systems AS src
    ON act.source_id = src.source_id;
```

Aliases

act = equipment_activity
src = source_systems

Qualified column names

act.status identifies which table owns the column.

Short aliases are good. Confusing aliases are not.

Rename Joined Output Columns

Use AS to create analysis-friendly column names

```
SELECT
    act.record_id,
    src.source_name AS source,
    src.department,
    act.status AS processing_status,
    act.record_count
FROM equipment_activity AS act
INNER JOIN source_systems AS src
    ON act.source_id = src.source_id;
```

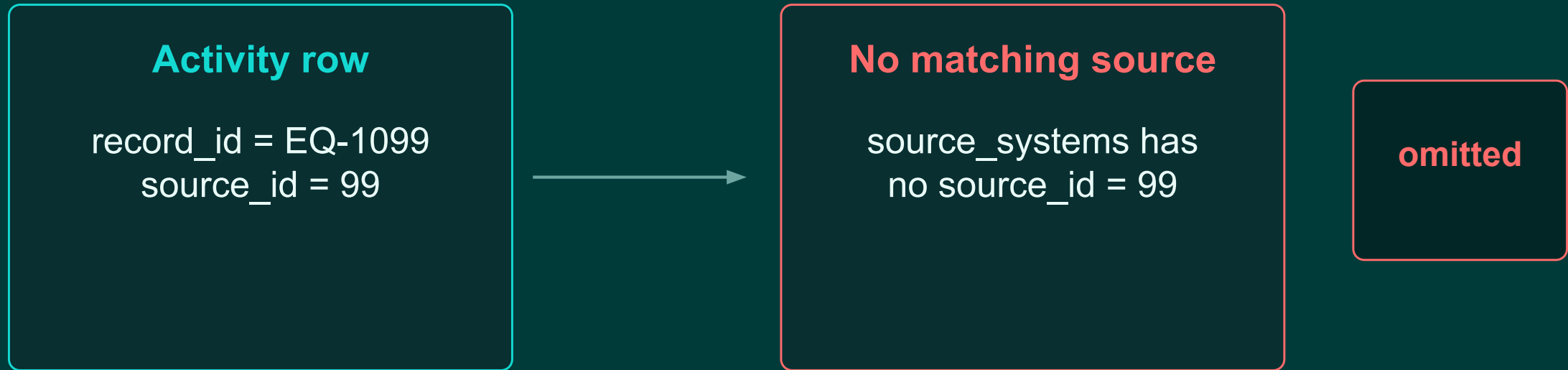
Result column names

```
record_id
source
department
processing_status
record_count
```

These names become pandas DataFrame columns when loaded with read_sql_query().

INNER JOIN Behavior

Unmatched rows disappear from the result



Great for matched analysis. Risky if you need to detect bad relationships.

LEFT JOIN

Keep every row from the left table

```
SELECT
    act.record_id,
    src.source_name,
    act.status,
    act.record_count
FROM equipment_activity AS act
LEFT JOIN source_systems AS src
    ON act.source_id = src.source_id;
```

Mental model

- Keep every activity row
- Attempt to match source rows
- If no match, source columns become NULL
- Unmatched rows remain visible

LEFT JOIN is usually better when reviewing data quality.

INNER JOIN Versus LEFT JOIN

Choose based on whether unmatched rows should be visible

INNER JOIN

Keep only rows that match in both tables.

matched rows only

LEFT JOIN

Keep every left-table row, with NULLs when no match exists.

good for data review

Practical question: should unmatched activity records disappear or be investigated?

Find Unmatched Activity Rows

Use LEFT JOIN plus IS NULL to detect missing relationships

```
SELECT
    act.record_id,
    act.source_id,
    act.status
FROM equipment_activity AS act
LEFT JOIN source_systems AS src
    ON act.source_id = src.source_id
WHERE src.source_id IS NULL;
```

Validation pattern

- Keep every activity row
- Try to match source metadata
- Select rows where the source side is NULL
- Investigate missing source records

This is a database version of finding missing values before analysis.

Filter Joined Results

Combine JOIN, WHERE, and ORDER BY

```
SELECT
    act.record_id,
    src.source_name,
    act.status,
    act.error_count
FROM equipment_activity AS act
INNER JOIN source_systems AS src
    ON act.source_id = src.source_id
WHERE src.source_name = 'maintenance'
    AND act.status = 'warning'
ORDER BY act.error_count DESC;
```

Full query pattern

```
SELECT columns
FROM primary table
JOIN related table
    ON relationship
WHERE conditions
ORDER BY result
```

SQL can join and reduce the data before pandas loads it.

SQL and pandas Have Different Strengths

Use the database for selection; use pandas for analysis

SQL is good for

- Selecting rows
- Selecting columns
- Joining related tables
- Reducing data before transfer

pandas is good for

- Cleaning values
- Calculated columns
- Aggregating for analysis
- Visualizing and reporting

These are overlapping tools, but this is a useful beginner division of labor.

Load SQL Results Directly into pandas

`pd.read_sql_query()` turns a query result into a DataFrame

```
import sqlite3
import pandas as pd

connection = sqlite3.connect(
    "equipment_activity.db"
)

records = pd.read_sql_query(
    query,
    connection,
)
```



Mental model

SQL query result



`pd.read_sql_query()`



DataFrame

The DataFrame columns come from the selected SQL columns and aliases.

Joined Query for pandas

Select useful columns and aliases before loading

```
query = """
SELECT
    act.activity_id,
    act.record_id,
    src.source_name AS source,
    src.department,
    act.status,
    act.record_count,
    act.completion,
    act.error_count,
    act.processed_date
FROM equipment_activity AS act
LEFT JOIN source_systems AS src
    ON act.source_id = src.source_id
"""
```

Why this query?

- Aliases create clear DataFrame columns
- LEFT JOIN preserves unmatched rows
- Only useful analysis fields are selected
- source and department are included

Avoid SELECT * in final joined analytical queries.

Inspect the Loaded DataFrame

LEFT JOIN NULL values become pandas missing values

```
records.head()  
  
records.info()  
  
records.isna().sum()
```

SQL NULL → pandas missing value

If a LEFT JOIN cannot find source metadata, source and department may be missing in the DataFrame.

Inspect after loading. Do not assume the joined dataset is complete.

Parse Dates and Pass Parameters

Keep SQL values separate from SQL syntax

```
query = """
SELECT
    act.record_id,
    src.source_name AS source,
    act.status,
    act.processed_date
FROM equipment_activity AS act
LEFT JOIN source_systems AS src
    ON act.source_id = src.source_id
WHERE act.processed_date >= ?
    AND act.status = ?
ORDER BY act.processed_date
"""
```

```
records = pd.read_sql_query(
    query,
    connection,
    params=(
        "2026-07-01",
        "warning",
    ),
    parse_dates=[
        "processed_date",
    ],
)
```

Same safety rule: use parameters instead of f-strings for query values.

Continue the Analysis in pandas

Once loaded, use familiar DataFrame skills

```
records_by_source = (  
    records.groupby("source")["record_count"]  
    .sum()  
    .sort_values(ascending=False)  
)  
  
warning_records = records.loc[  
    records["status"] == "warning"  
]  
  
records["remaining_percentage"] = (  
    100 - records["completion"]  
)
```

Integrated workflow

SQLite
↓
SQL SELECT + JOIN
↓
pandas DataFrame
↓
clean, analyze, visualize

SQL created the useful dataset. pandas answers the analytical questions.

Load Only What the Notebook Needs

Close the connection after the DataFrame is loaded

Less useful

Load every row
Load every column
Filter much later

Better when requirements are clear

SELECT useful columns
WHERE useful rows
JOIN useful tables

```
connection.close()  
  
# DataFrame remains available in memory.
```

Changing the DataFrame does not automatically update the database.

Guided Exercise Part 1: Joins

Inspect tables, join rows, and compare join behavior

```
SELECT
    activity.record_id,
    sources.source_name,
    activity.status,
    activity.record_count
FROM equipment_activity AS act
INNER JOIN source_systems AS src
    ON act.source_id = src.source_id
ORDER BY act.record_id;
```

Tasks

- Inspect source_systems
- Inspect equipment_activity
- Identify primary and foreign keys
- Create an INNER JOIN
- Create a LEFT JOIN
- Compare row counts

Time box: 14 minutes

Explain why INNER JOIN and LEFT JOIN can return different numbers of rows.

Guided Exercise Part 2: Load into pandas

Use `read_sql_query()`, parse dates, and analyze the result

```
records = pd.read_sql_query(
    query,
    connection,
    parse_dates=[
        "processed_date",
    ],
)

records.head()
records.info()
records.isna().sum()
```

Analysis questions

- How many rows belong to each source?
- Which source has greatest total record_count?
- How many rows have missing source information?
- What is average completion by department?

Time box: 16 minutes

Objective: SQL selects and joins; pandas analyzes.

Applied Challenge: SQLite to pandas Report

Build a notebook section that joins, loads, analyzes, visualizes, and explains

Connect

JOIN query

LEFT JOIN

read_sql_query()

parse
dates

inspect
missing

summarize

chart

close

Required notebook actions

- Load a joined DataFrame
- Preserve unmatched activity rows
- Summarize total record_count by source
- Summarize average completion by department
- Identify missing source relationships

Markdown explanations

- What the join accomplished
- Which work happened in SQL
- Which work happened in pandas
- What the chart supports

Final output: an analysis-ready SQLite-to-pandas notebook report section.

Knowledge Check and Wrap-Up

Students should understand the SQLite-to-pandas workflow

Why store source details in a separate table?

What is a foreign key?

What does a join do?

What does INNER JOIN keep?

What does LEFT JOIN keep?

How can unmatched rows be detected?

What does `pd.read_sql_query()` return?

Why parse `processed_date` while loading?

Final idea: SQLite stores related tables; SQL joins useful data; pandas continues the analysis.